## PRACTICAL NO -1

**1. Create different blinking patterns with LEDs by connecting to GPIO pints of Raspberry Pi.**

**Code:**

```
from gpiozero import LED
import time

led1 = LED(17)
led2 = LED(18)
led3 = LED(27)
led4 = LED(22)

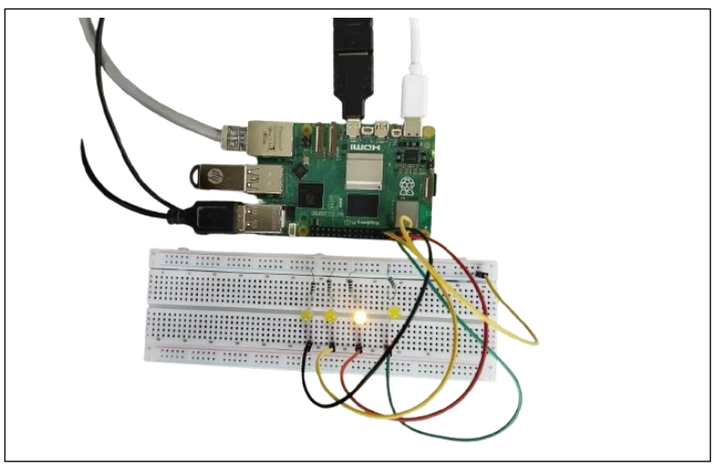
while(True):
    led1.on()
    time.sleep(0.5)
    led1.off()

    led2.on()
    time.sleep(0.5)
    led2.off()

    led3.on()
    time.sleep(0.5)
    led3.off()

    led4.on()
    time.sleep(0.5)
    led4.off()
```

**Output:**



## PRACTICAL NO -2

**2. Use Telegram to control Raspberry Pi GPIO.**

**Code:**

```
import telepot
import time
import sys
from gpiozero import LED

def handle(msg):
    command = msg['text']

    if(command == '/on_led1'):
        led1.on()
    elif(command == '/off_led1'):
        led1.off()
    elif(command == '/on_led2'):
        led2.on()
    elif(command == '/off_led2'):
        led2.off()

bot = telepot.Bot(8872858696:AAEMi6aZ6XlFdwtE4NlzbUq1FeizLR4-L7c')
bot.message_loop(handle)
print('bot is active')

led1 = LED(17)
led2 = LED(18)

led1.off()
led2.off()
```

**Output:**



## PRACTICAL NO -3

**3. Interface TM1637 with Raspberry Pi and Display current time.**
**Code:**

```
import time

import tm1637

Display=tm1637.TM1637(CLK=21,DIO=20,brightness=1)

colon_state = True

while True:

    current_time = time.localtime()

    hours = current_time.tm_hour

    minutes = current_time.tm_min

    digit1 = hours // 10

    digit2 = hours % 10

    digit3 = minutes // 10

    digit4 = minutes % 1

    time_digits = [digit1, digit2, digit3, digit4]

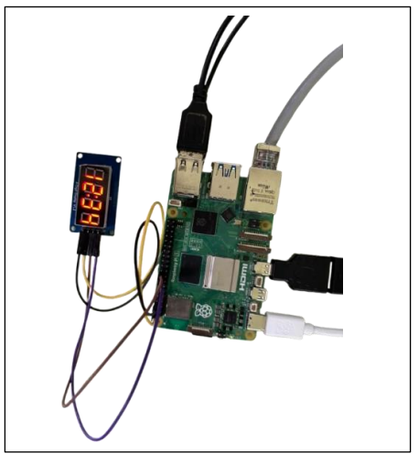
    Display.Show(time_digits)

    Display.ShowDoublepoint(colon_state)

    colon_state = not colon_state

    time.sleep(1.0)
```

**output:**



- **Increase and Decrease Brightness of Digit**

**Code:**

```
import time
import tm1637

Display = tm1637.TM1637(clk=23, dio=24)

digits = [1, 2, 3, 4]
Display.Show(digits)

time.sleep(5)
Display.Clear()

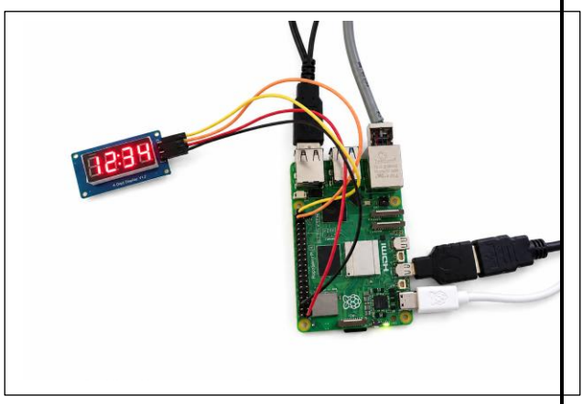
Display.Show1(0, 1)
time.sleep(1)
Display.Show1(1, 2)
time.sleep(1)
Display.Show1(2, 3)
time.sleep(1)
Display.Show1(3, 4)
time.sleep(1)

Display.ShowDoublepoint(True)
time.sleep(5)

Display.SetBrightness(0)
time.sleep(1)
Display.SetBrightness(0.2)
time.sleep(1)
Display.SetBrightness(0.4)
time.sleep(1)
Display.SetBrightness(0.6)
time.sleep(1)
Display.SetBrightness(0.8)
time.sleep(1)
Display.SetBrightness(1)
time.sleep(5)

Display.SetBrightness(0.8)
time.sleep(1)
Display.SetBrightness(0.6)
time.sleep(1)
Display.SetBrightness(0.4)
time.sleep(1)
Display.SetBrightness(0.2)
time.sleep(1)
Display.SetBrightness(0)
time.sleep(1)
Display.Clear()
```

**Output:**




## PRACTICAL NO -4

**4. Interface GPS module with Raspberry Pi.**

**Code:**

## PRACTICAL NO -5

**5. Interface ADS 1115 with Raspberry Pi to act as oscilloscope.**
**Code:**

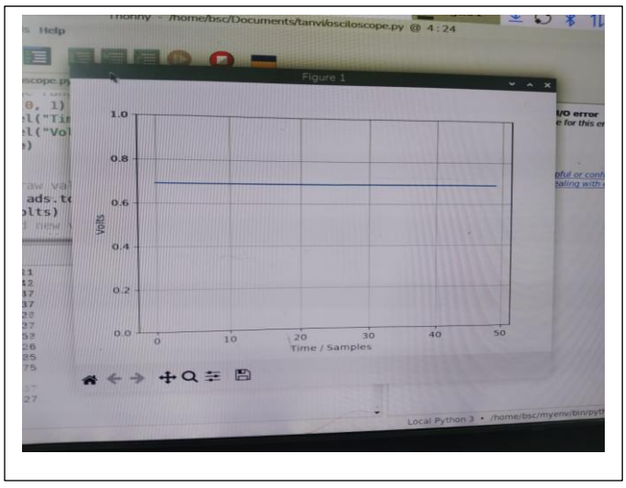
```
from collections import deque
from ADS1x15 import ADS1115
import matplotlib
matplotlib.use("TkAgg")
import matplotlib.pyplot as plt

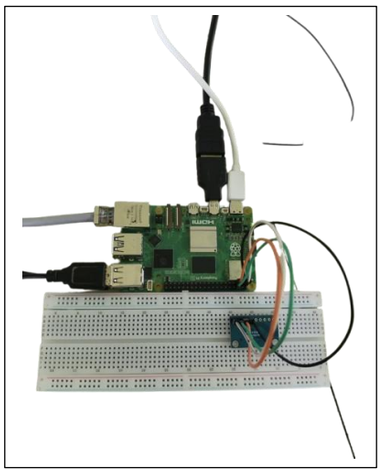
ads = ADS1115(1, 0x48)
data = deque([0.0] * 50, maxlen=50)

plt.ion()
fig, ax = plt.subplots()
(line,) = ax.plot(data)
ax.set_ylim(0.276, 0.278)
ax.set_xlabel("Time / Samples")
ax.set_ylabel("Volts")
ax.grid(True)

while True:
    volts = ads.toVoltage(ads.readADC(0))
    print(volts)
    data.append(volts)
    line.set_ydata(data)
    plt.pause(0.05)
```

**output:**



## PRACTICAL NO -6

**6. Interface 16x2 LCD with Raspberry Pi using I2C module to display text.**
**Code:**

```
from RPLCD.i2c import CharLCD
import time

# Initialize with explicit backlight control
lcd = CharLCD(
    i2c_expander='PCF8574',
    address=0x27,
    backlight_enabled=True
)

# clear screen
lcd.clear()

# write content
lcd.write_string('Prachi')

# move cursor to 2nd row, first column
lcd.cursor_pos = (1, 0)

# write content
lcd.write_string('T.Y.I.T')
```

**Output:**



## PRACTICAL NO -7

**7. Interface RFID module with Raspberry Pi to read and write to RFID tags/cards.**
**Code:**

**WriteRFID.py**

```
from rfid_pi5 import SimpleRC522

rfid = SimpleRC522()

text = input("Enter text : ")
print("Hold your tag near the reader...")

result = rfid.write(text)

if result == True:
    print("Successfully written to tag!")
else:
    print("Failed to write.")
```

**ReadRFID.py**

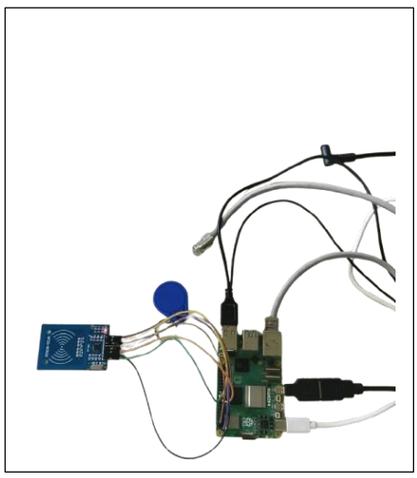
```
from rfid_pi5 import SimpleRC522

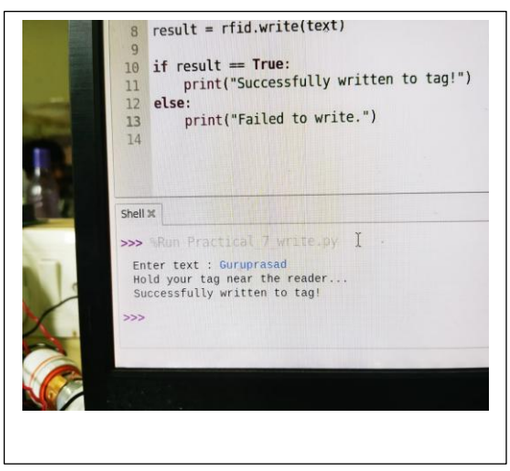
rfid = SimpleRC522()

print("Hold your tag near the reader to scan...")
uid, text = rfid.read()

if uid:
    print(f"Card UID : {uid}")
    print(f"Data Read: '{text}'")
else:
    print("No card detected within timeout.")
```

output:





**Practical no 8**

**8. Interface Fingerprint Sensor with Raspberry Pi.**

**Code:**

```
import time
from pyfingerprint.pyfingerprint import PyFingerprint

f = PyFingerprint('/dev/ttyAMA0', 57600)

def register_finger():
    print("Place finger to register...")
    while not f.readImage(): pass
    f.convertImage(1)

    print("Remove finger...")
    time.sleep(2)

    print("Place same finger again...")
    while not f.readImage(): pass
    f.convertImage(2)

    print("Remove finger...")
    time.sleep(2)

    f.createTemplate()
    f.storeTemplate(1)
    print("Saved successfully to Slot 1!")

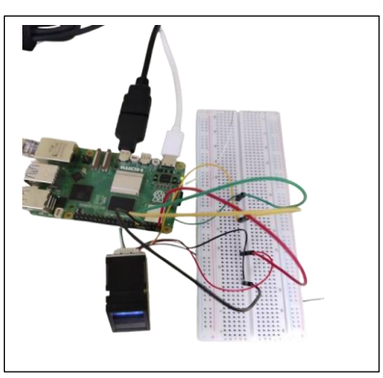
def login_finger():
    print("Place finger to login...")
    while not f.readImage(): pass
    f.convertImage(1)

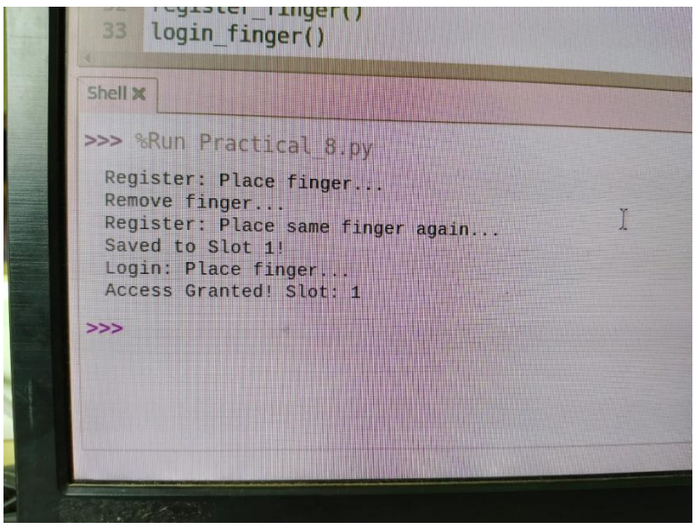
    position, _ = f.searchTemplate()

    if position >= 0:
        print(f"Access Granted! Slot: {position}")
    else:
        print("Access Denied!")

register_finger()
login_finger()
```

**output:**




33 login_finger()
Shell
>>> %Run Practical_8.py
Register: Place finger...
Remove finger...
Register: Place same finger again...
Saved to Slot 1!
Login: Place finger...
Access Granted! Slot: 1
>>>

## PRACTICAL NO -9

**9. Interface Pi Camera with Raspberry Pi to capture image and video.**
**Code:**

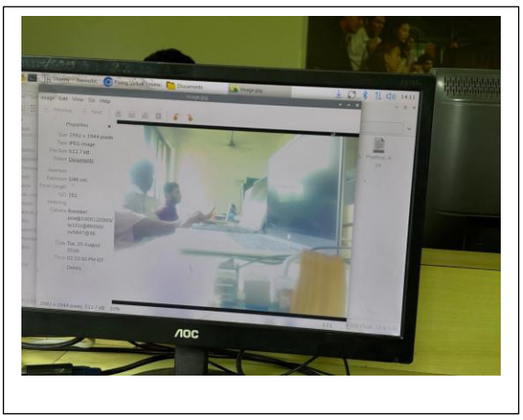
```
from picamera2 import Picamera2

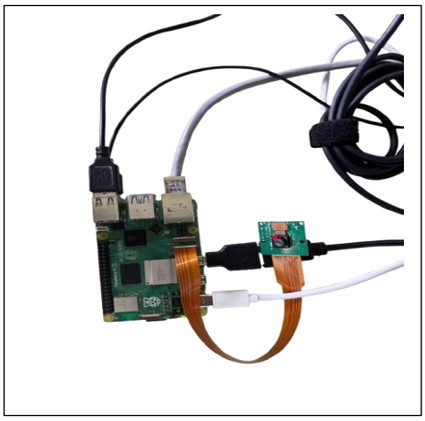
picam = Picamera2()


picam.start_and_capture_file( "image.jpg")


picam.start_and_record_video("practical_video.mp4", duration=5)
```

**output:**



## PRACTICAL NO -10

**10. Design IoT based home automation system using Raspberry Pi**
**Code:**

```
from gpiozero import OutputDevice
import time

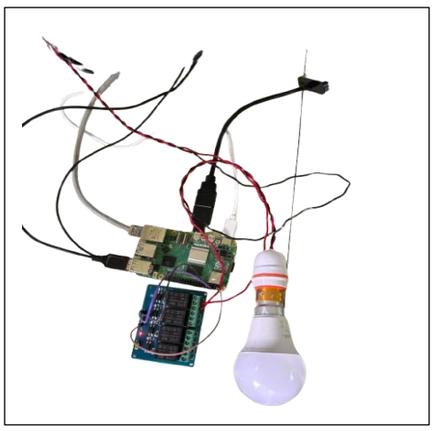
relay = OutputDevice(17, active_high=False, initial_value=False)

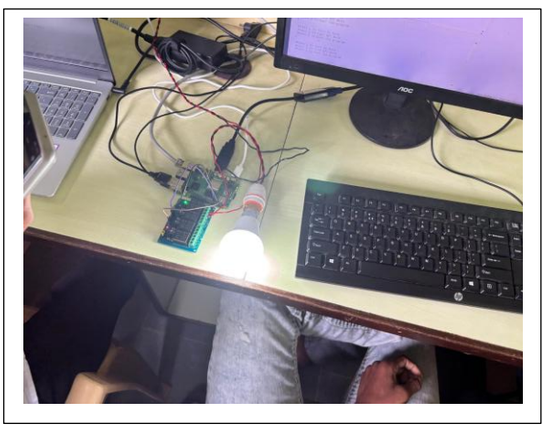
while(True):
    n = int(input("\n Press 1 to Turn On Bulb \n Press 2 to Turn Off Bulb \n Press 3 to exit the program \n"))

    if(n == 1):
        relay.on()
    elif(n == 2):
        relay.off()
    elif(n == 3):
        relay.off()
        break;
    else:
        print("Invalid Input")

print("Program Exited")
```

**output:**





**PRACTICAL NO -11**

**11. Interface Fingerprint Sensor and 16X2 LCD with Raspberry and display message RFID UID matched or unmatched.**

**Code:**

**ReadRFID.py**

```
from RPLCD.i2c import CharLCD

import time
from rfid_pi5 import SimpleRC522
rfid = SimpleRC522()

print("Hold your tag near the reader to scan...")
uid, text = rfid.read()

lcd = CharLCD(
    i2c_expander = 'PCF8574',
    address = 0x27,
    backlight_enabled = True
    )
lcd.clear()

if(text == 'Guru'):
    lcd.write_string('Login Successfull')
else:
    lcd.write_string('Login Failed')
```

**WriteRFID.py**

```
from rfid_pi5 import SimpleRC522

rfid = SimpleRC522()

text = input("Enter text:")

print("Hold your tag near the reader...")

result = rfid.write(text)

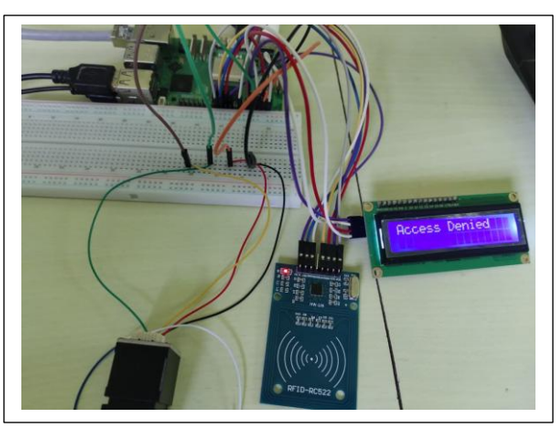
if result == True:

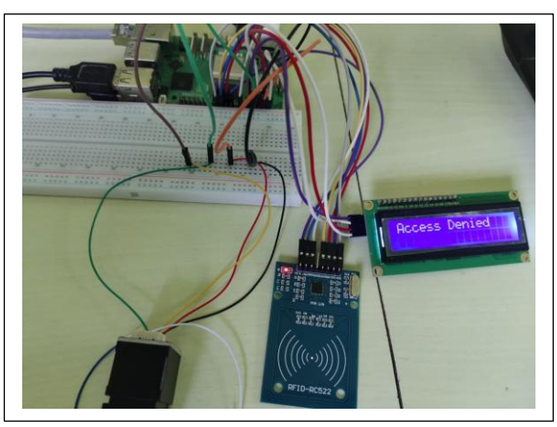
    print("Successfully written to tag! ")

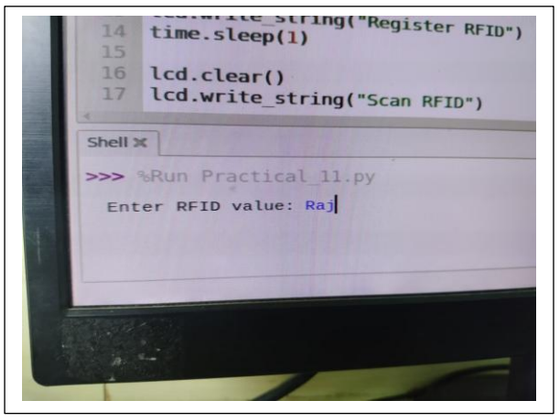
else:

    print("Failed to write")
```

**output:**




14 time.sleep(1)
15
16 lcd.clear()
17 lcd.write_string("Scan RFID")
Shell
>>> %Run Practical_11.py
Enter RFID value: Raj

**PRCATICAL NO -12**

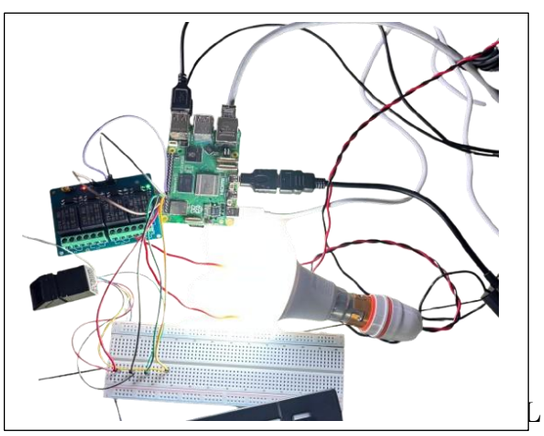
**12. Simulate door lock by interfacing fingerprint sensor and relay moule with raspberry Pi.**
**Code:**

```
from gpiozero import OutputDevice
import time
from pyfingerprint.pyfingerprint import PyFingerprint
relay = OutputDevice(17, active_high=False, initial_value=False)
f = PyFingerprint('/dev/ttyAMA0', 57600)
def register_finger():
    print("Place finger to register...")
    while not f.readImage(): pass
    f.convertImage(1)
    print("Remove finger...")
    time.sleep(2)
    print("Place same finger again...")
    while not f.readImage(): pass
    f.convertImage(2)

    print("Remove finger...")
    time.sleep(2)
    f.createTemplate()
    f.storeTemplate(1)
    print("Saved successfully to Slot 1!")
def login_finger():
    print("Place finger to login...")
    while not f.readImage(): pass
    f.convertImage(1)
    position, _ = f.searchTemplate()
    if position >= 0:
        relay.on()

        print(f"Access Granted! Slot: {position}")
    else:
        print("Access Denied!")
register_finger()
login_finger()
```

**output:**



**L NO -13**

**13. Implement basic Publisher – Subscriber using MQTT. Publish messages using paho mqtt. Subscribe and control LED.**

Run Subscriber.py using Terminal

- sudo systemctl start mosquitto
- sudo systemctl enable mosquitto
- source myenv/bin/activate
- cd **/home/bsc/**programDirectory
- python3 Subscriber.py

**Subscriber.py**

```
from gpiozero import LED
import paho.mqtt.client as mqtt

led = LED(18)

def on_message(client, userdata, msg):
    cmd = msg.payload.decode().strip().upper()
    if(cmd == "ON"):
        led.on()
    else:
        led.off()

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_message = on_message
client.connect("localhost", 1883)
client.subscribe("home/pi/led")
client.loop_forever()
```

**Publisher.py**

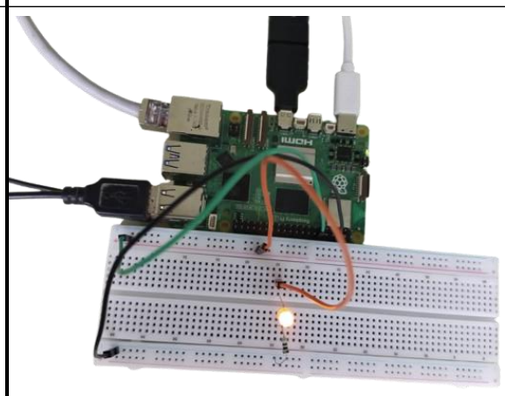
```
import paho.mqtt.client as mqtt
import time

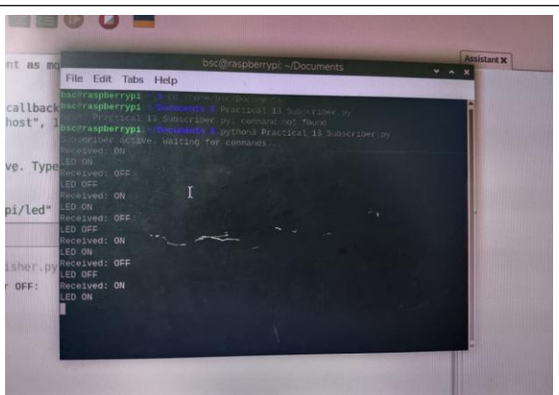
client = mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.connect("localhost", 1883)
client.loop_start()

print("Publisher active. Type ON or OFF:")

while True:
    endpoint = "home/pi/led"
    msg = input("Enter Command (ON/OFF): ").strip()
    client.publish(endpoint, msg)
```

**Output-**





**PRACTICAL NO -14**

**14. Design a web dashboard using Flask. Control relay module via web.**

**Code:**

**File Structure**

```
my_project/
├── app.py
└── templates/
    └── index.html
```

**app.py**

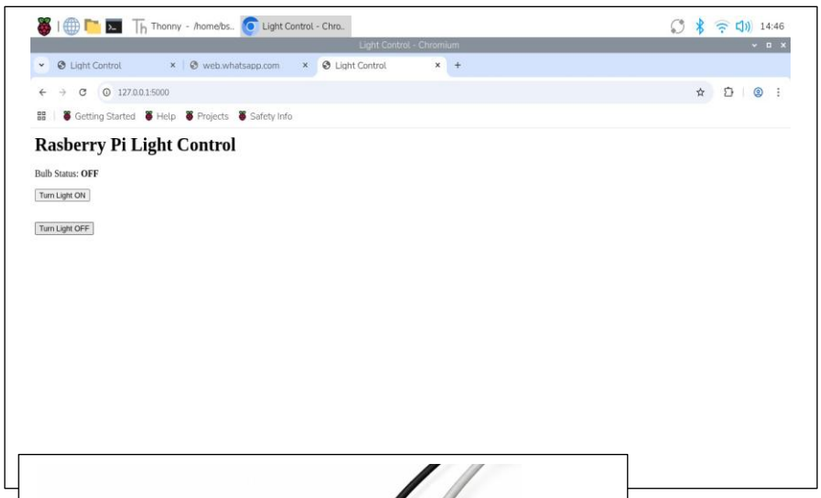
```
from flask import Flask, render_template, redirect, url_for
from gpiozero import OutputDevice
app = Flask(__name__)
relay = OutputDevice(17, active_high=False, initial_value=False)
current_state = "OFF"
@app.route('/')
def home():
    return render_template('index.html', status=current_state)
@app.route('/on', methods=['POST'])
def turn_on():
    global current_state
    relay.on()
    current_state = "ON"
    return redirect(url_for('home'))
@app.route('/off', methods=['POST'])
def turn_off():
    global current_state
    relay.off()
    current_state = "OFF"
    return redirect(url_for('home'))
app.run(host='0.0.0.0', port=5000
```

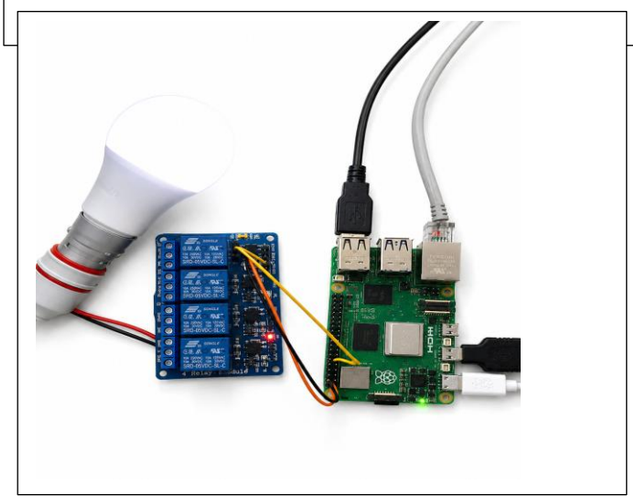
**index.html**

```
<html>
<head>
    <title>Light Control</title>
</head>
<body>
    <h1>Raspberry Pi Light Control</h1>
    <p>Bulb Status: <strong>{{ status }}</strong></p>

    <form action="/on" method="POST">
        <button type="submit">Turn Light ON</button>
    </form>
    <br>
    <form action="/off" method="POST">
        <button type="submit">Turn Light OFF</button>
    </form>
</body>
</html>
```

## Output:


Thonny - /home/bs..
Light Control - Chro..
14:46
Light Control - Chromium
Light Control
web.whatsapp.com
Light Control
127.0.0.1:5000
Getting Started
Help
Projects
Safety Info
Rasberry Pi Light Control
Bulb Status: OFF
Turn Light ON
Turn Light OFF



**PRACTICAL NO -15**

**15. Create two factor authentication system by interfacing fingerprint sensor, RFID and relay with Raspberry Pi.**
**Code:**

```
from RPLCD.i2c import CharLCD
from rfid_pi5 import SimpleRC522
from gpiozero import OutputDevice
import time
from pyfingerprint.pyfingerprint import PyFingerprint
rfid_matched = False
finger_matched = False
relay = OutputDevice(17, active_high=False, initial_value=False)
f = PyFingerprint('/dev/ttyAMA0', 57600)
def register_finger():
    print("Place finger to register...")
    while not f.readImage(): pass
    f.convertImage(1)
    print("Remove finger...")
    time.sleep(2)
    print("Place same finger again...")
    while not f.readImage(): pass
    f.convertImage(2)

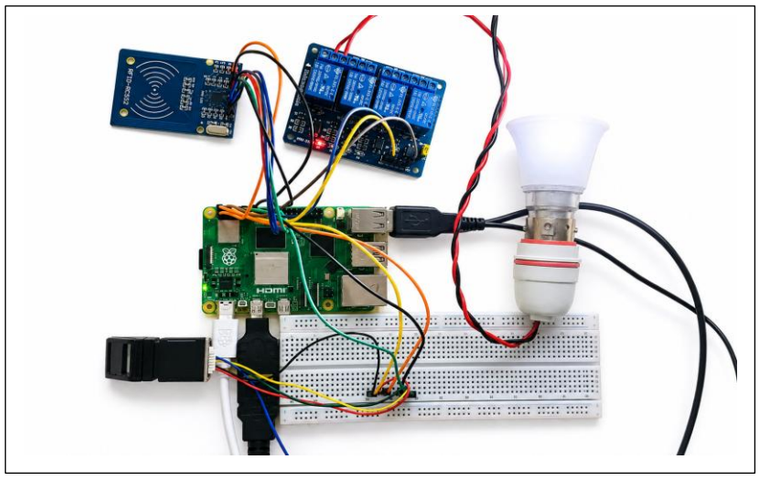
    print("Remove finger...")
    time.sleep(2)
    f.createTemplate()
    f.storeTemplate(1)
    print("Saved successfully to Slot 1!")
def login_finger():
    print("Place finger to login...")
    while not f.readImage(): pass
    f.convertImage(1)
    position, _ = f.searchTemplate()
    global finger_matched
    if position >= 0:
        print('finger matched')
        finger_matched = True
    else:
        print('finger not matched')
        finger_matched = False

def login_rfid():
    rfid = SimpleRC522()

    print("Hold your tag near the reader to scan...")
    uid, text = rfid.read()
    global rfid_matched
    if(text == 'Aatiti'):
        print('rfid matched')
        rfid_matched = True
    else:
        print('rfid not matched')
        rfid_matched = False
register_finger()
login_finger()
login_rfid()
if(rfid_matched == True and finger_matched == True):
    relay.on()
    time.sleep(5)
    relay.off()
```

```
    print('Login Successfull')
else:
    relay.off()
    print('Login Failed')
```

**Output:**

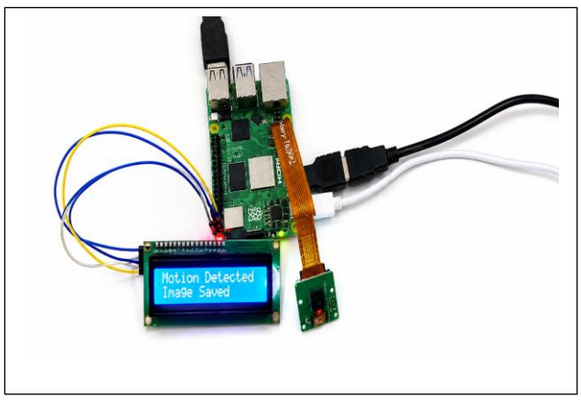


## PRACTICAL NO 16

**16. Create a motion detector by interfacing camera and 16X2 display with Raspberry Pi. Print the appropriate message on the display.**
**Code:**

```
import time
import numpy as np
from picamera2 import Picamera2
picam = Picamera2()
picam.start()
prev = picam.capture_array()[:, :, 1].astype(int)
print("Camera Motion Detector is Active")
while True:
    curr = picam.capture_array()[:, :, 1].astype(int)
    diff =np.abs(prev - curr)
    thresh = diff > 2
    changed_pixels = np.sum(thresh)
    if changed_pixels > 3000:
        fileName = f"img_{time.time()}.jpg"
        picam.capture_file(fileName)
        print("Motion detected! Saved:", fileName)
        time.sleep(2)
    prev = curr
    time.sleep(0.1)
```

**Output:**



## PRACTICAL NO -17

**17. Design and implement a Node-RED flow on Raspberry Pi that integrates a physical push-button and an LED with a web dashboard. The flow must allow users to toggle the LED from the dashboard and via the physical button. A button press should force the LED ON for 5 seconds and then automatically restore the LED to the dashboard's last state.**
**Code:**

1. **Dashboard Switch (ui-switch):**
   - **Topic:** dashboard_state
   - **Output:** true (boolean) / false (boolean)
2. **Change Node (change):**
   - **Rule:** Set global.lastState = msg.payload
3. **GPIO In Node (rpi-gpio in):**
   - **Pin:** GPIO23 - (Pin 16)
   - **Resistor:** Pullup
   - **Debounce:** 50 ms
4. **Function Node (function 1):**
   - Write JavaScript logic:
5. **GPIO Out Node (rpi-gpio out):**
   - **Pin:** GPIO18 - (Pin 12)
   - Connect outputs from both the **Change Node** and **Function Node** into this single GPIO Out node.

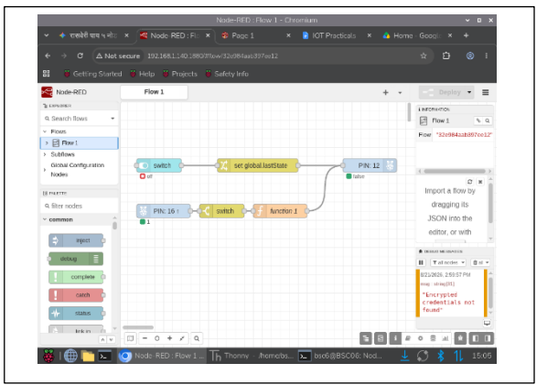
**Deploy & Test**

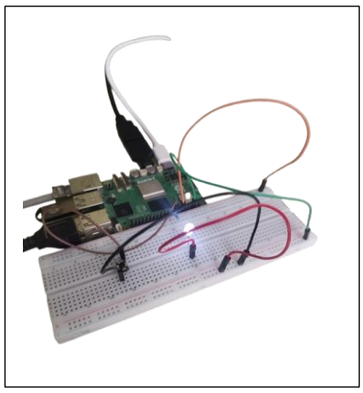
1. Click the red **Deploy** button in the top-right corner.
2. Open Dashboard 2.0 in your browser (http://<your-pi-ip>:1880/dashboard)

**JavaScript Code**

```
if (msg.payload === 0 || msg.payload === false) {
    node.send({ payload: true });
    setTimeout(function() {
        var lastState = global.get('lastState') || false;
        node.send({ payload: lastState });
    }, 5000);
}
return null;
```

**output:**



**18. Write a program to read the CPU%, Memory% and CPU temperature of Raspberry Pi.**

**Code:**

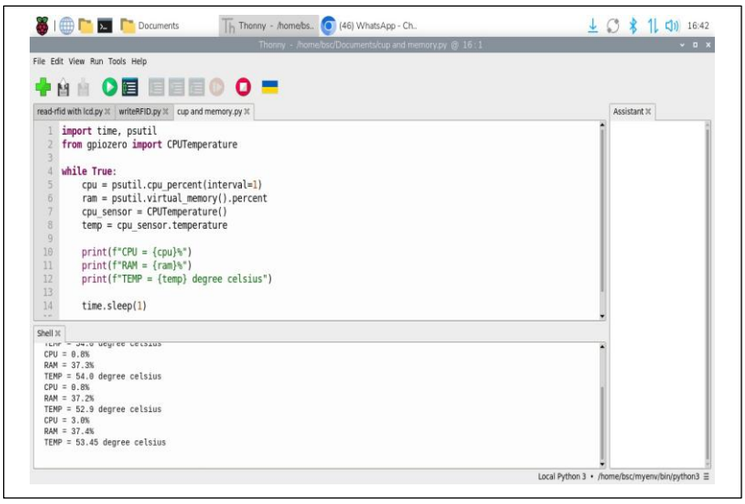
```
import time, psutil
from gpiozero import CPUTemperature

while True:
    cpu = psutil.cpu_percent(interval=1)
    ram = psutil.virtual_memory().percent
    cpu_sensor = CPUTemperature()
    temp = cpu_sensor.temperature

    print(f"CPU = {cpu}%")
    print(f"RAM = {ram}%")
    print(f"TEMP = {temp} degree celsius")

    time.sleep(1)
```

**Output:**



**PRACTICAL NO -19**

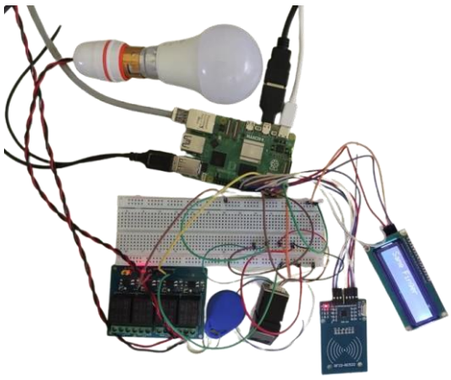
**19. Design and implement an embedded access-control prototype that grants entry only when either a whitelisted RFID tag is detected or a legitimate fingerprint is verified, provides real-time visual feedback on an LCD, indicates status via an LED, and pulses a relay (active-LOW) to unlock for a fixed interval. The system must log all events (time, method, result) for auditability.**

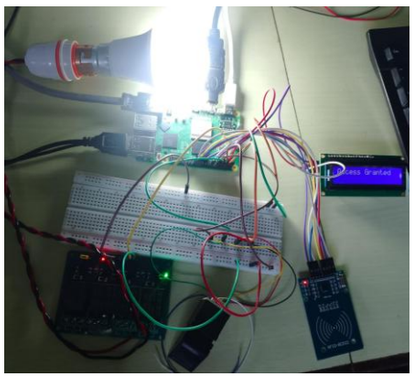
**Code:**

```
from gpiozero import OutputDevice
import time
from pyfingerprint.pyfingerprint import PyFingerprint
relay = OutputDevice(17, active_high=False, initial_value=False)
f = PyFingerprint('/dev/ttyAMA0', 57600)
def register_finger():
    print("Place finger to register...")
    while not f.readImage(): pass
    f.convertImage(1)
    print("Remove finger...")
    time.sleep(2)
    print("Place same finger again...")
    while not f.readImage(): pass
    f.convertImage(2)

    print("Remove finger...")
    time.sleep(2)
    f.createTemplate()
    f.storeTemplate(1)
    print("Saved successfully to Slot 1!")
def login_finger():
    print("Place finger to login...")
    while not f.readImage(): pass
    f.convertImage(1)
    position, _ = f.searchTemplate()
    if position >= 0:
        relay.on()
        time.sleep(5)
        relay.off()
        print(f"Access Granted! Slot: {position}")
    else:
        print("Access Denied!")
register_finger()
login_finger()
```

**output:**




Access Granted